

1. Scenario : parsing Table Conflict in LL(1)

Sol:

An LL(1) parser is top down parser that reads the input from left to right and produces a left most derivation using one look ahead symbol. The parsing table is constructed using the first and follow sets.

First set:- Contains the terminal that can appear at the beginning of string derived from a non terminal.

Follow set:- Contains the terminals that can appear immediately after a non terminal.

A parsing table conflict occurs when more than once production is entered into the same table cell for a non-terminal and a input symbol.

- left recursion
- left factoring problems
- Ambiguous grammar

Decision

To make the grammar suitable for LR(1) parsing
eliminate left recursion
Perform left factoring
Ensure first and follow sets do not overlap
After these modifications each parsing table
entry contains any production resulting from
LR(1) and conflict free parsing.

2. Scenario : incorrect parse Tree generation

Sol:- A syntax analyzer constructs a parse
tree by applying grammar production according
to the input tokens the parse tree represents
the syntactic structure of the expression.

For the Expression

$id + id - id$

The order of evaluation depends on the
structures of the parse tree.

if the grammar does not define associa-
tivity the parser may generate the parse
tree as:

$id + (id - id)$
instead of
 $(id + id) - id$

Since, addition and subtraction are left-association the second interpretation is correct.

Decision:-

modify the grammar to enforce left associativity

grammar

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow id$

This - grammar ensures expressions are evaluated from left to right producing the correct parse tree and correct evaluation.

③ Scenario invalid Expression Handling.

The lexical analyzer converts the input

$a + a b$

into valid tokens

identifier(a)

operator($+$)

operator($*$)

Identifier(b)

After the "+" operator the grammar expects an operand (identifier or Expression) instead another operator "*" appears.

Therefore the parser cannot construct a valid parse tree and reports a syntax error.

in predictive parsing (LR parser).

- The parser checks the parsing table.

- Since no valid production exists it reports an error.

- Shift-Reduce parsing (LR parser)

Decision

To improve parsing

- use parse mode Recovery.
- use phrase-level Recovery.
- use Synchronizing Tokens based on follow set.

4. Scenario = Ambiguity in Conditional statement

Sol:

Given grammar;

$S \rightarrow \text{if } \epsilon \text{ then } S$

$S \rightarrow \text{if } \epsilon \text{ then } S \text{ else } S$

$S \rightarrow \epsilon$

This grammar is ambiguous because the parser cannot determine to which of statement the else belongs.

This ambiguity to which of statement the else belongs.

Both productions being with the same prefix it is then

5. Scenario: operators precedence Conflict

* Given Grammar

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

This grammar is ambiguous because it does not define operator precedence or associativity

for the Expression.

$$id + id * id$$

Two parse trees are possible.

1. $(id + id) * id$

2. $id + (id * id)$

The Correct Evaluation is;

$$id + (id * id).$$

Because multiplication has higher precedence than addition.

Without precedence.

During parsing the parser cannot decide neither to:

Reduce using $s \rightarrow it \in \text{then } s$

Continue using $s \rightarrow it \in \text{then } s \text{ else}$

This executes parsing conflicts and multiple possible parser trees.

Decision

~~~~~  
~\*~

Recursion the grammar using matched and unmatched statements.

$s \rightarrow M / o$

$M \rightarrow it \in, \text{then } M \text{ else } M / a$

$U \rightarrow it \in \text{then } s$

,  $it \in \text{then } M \text{ else } v$

- This grammar remove ambiguity

Another common solution used by programming language is

Associate every class with the nearest unmatched  $it$ ,